Systems Engineering

WILEY

REVIEW ARTICLE

A Conceptual Introduction to Hetero-Functional Graph Theory for Systems-of-Systems

Amro M. Farid[1,2] | Amirreza Hosseini[1] | John C. Little[3]

[1]Department of Systems Engineering, Charles V. Schaefer, Jr. School of Engineering and Science, Stevens Institute of Technology, Hoboken, New Jersey, USA | [2]MIT Mechanical Engineering, Cambridge, Massachusetts, USA | [3]Charles E. Via, Jr. Department of Civil and Environmental Engineering, College of Engineering, Virginia Tech, Blacksburg, Virginia, USA

Correspondence: Amirreza Hosseini (shossein1@stevens.edu)

Received: 27 October 2025 | **Revised:** 6 March 2026 | **Accepted:** 27 March 2026

Keywords: convergence | hetero-functional graph theory | model-based systems engineering | network science | ontology

ABSTRACT

A defining feature of 21st century engineering challenges is their inherent complexity, demanding the convergence of knowledge across diverse disciplines. Establishing consistent methodological foundations for engineering systems remains a challenge—one that both systems engineering and network science have sought to address. Model-based systems engineering (MBSE) has recently emerged as a practical, interdisciplinary approach for developing complex systems from concept through implementation. In contrast, network science focuses on the quantitative analysis of networks present within engineering systems. This article introduces hetero-functional graph theory (HFGT) as a conceptual bridge between these two fields, serving as an entry point for both communities. For systems engineers, HFGT preserves the heterogeneity of conceptual and ontological constructs in MBSE, including system form, function, and concept. For network scientists, it provides multiple graph-based data structures enabling matrix-based quantitative analysis. The modeling process begins with ontological foundations, where an engineering system is defined as an abstraction and represented with a model. Model fidelity is assessed using four linguistic properties: soundness, completeness, lucidity, and laconicity. A meta-architecture is introduced to manage the convergence challenges between domain-specific reference architectures and case-specific instantiations. Unlike other meta-architectures, HFGT is rooted in linguistic structures, modeling resources as subjects, system processes as predicates, and operands—such as matter, energy, organisms, information, and money—as objects. These elements are integrated within a system meta-architecture expressed in the Systems Modeling Language (SysML). The article concludes by offering guidance for further reading.

1 | Introduction: Motivation for Hetero-Functional Graph Theory (HFGT)

One defining characteristic of 21st century engineering challenges is the breadth of their scope. These societal challenges include:

1) Stabilize carbon emissions
2) Manage the nitrogen cycle
3) Provide access to clean water
4) Adapt to climate change
5) Provide infrastructure for urbanized populations
6) Feed a growing population sustainably
7) Manage the phosphorus cycle
8) Clean the world's oceans of solid waste
9) Restore and improve global biodiversity
10) Supply human needs for energy sustainably

Significance and Practitioner Points

This article introduces hetero-functional graph theory (HFGT) as a methodological bridge between the graphical modeling of Model-Based Systems Engineering (MBSE) and mathematical models founded in network science, dynamic systems, and operations research. For researchers, HFGT provides a rigorous methodological foundation that addresses the complex and heterogeneous interdependencies in systems of systems, overcomes the ontological limitations of multilayer networks, and reconciles the methods for structural analysis, dynamic simulation, and optimization. It addresses a critical gap in the systems science literature by enabling the mathematical synthesis of structural and functional viewpoints of complex engineering systems. For practitioners, HFGT offers a scalable and automatable methodology to translate complex architectural diagrams into computable mathematical models. This allows for proactive and self-consistent design and analysis of the structural and dynamic properties of a system-of-systems from the earliest phases of the system life cycle to its final implementation. Ultimately, the paper provides a conceptual introduction to HFGT, demonstrating a foundational language for engineering systems that ensures architectural descriptions are not just visual aids, but mathematically actionable blueprints.

At first glance, each of these aspirational engineering goals is so large and complex in its own right that it might seem entirely intractable. To adapt to each compelling challenge, we must converge knowledge that collectively spans every discipline. And yet, these seemingly disconnected challenges are actually intertwined in Systems-of-Systems. A convergent initiative addressing a single societal challenge is likely to be uncoordinated with, and perhaps counterproductive to, another convergent effort targeting a different societal challenge. Our single-discipline, single-problem thinking must evolve into a multi-discipline, multiproblem approach. Fortunately, a developing consensus is emerging across several STEM (science, technology, engineering, and mathematics) fields, indicating that each of these goals is characterized by an "engineering system" that is analyzed and re-synthesized using a convergence paradigm comprising a meta-problem-solving skill set. Instead of identifying a plethora of 21st-century societal challenges, it may be valuable to reformulate these challenges into a single convergence challenge that advances a complex system-of-systems [1].

Definition 1. Engineering system [2]: A class of systems characterized by a high degree of technical complexity, social intricacy, and elaborate processes, aimed at fulfilling essential functions in society.

The challenge of convergence towards abstract and consistent methodological foundations for engineering systems is formidable. Consider the engineering systems taxonomy presented in Table 1. It classifies engineering systems by five generic functions that fulfill human needs: (1) transform; (2) transport; (3) store; (4) exchange; and (5) control. On another axis, it classifies them by their operands: (1) living organisms (including people); (2) matter; (3) energy; (4) information; and (5) money. This classification presents a broad array of application domains that must be consistently treated. Furthermore, these engineering systems are at various stages of development and will likely remain so for decades, if not centuries. And so, the study of engineering systems must equally support design synthesis, analysis, and re-synthesis, while fostering innovation, whether incremental or disruptive.

Two fields in particular have attempted to traverse this convergence challenge: systems engineering and network science. Systems engineering, and more recently model-based systems engineering (MBSE), has evolved as a practical and interdisciplinary engineering discipline that enables the successful realization of complex systems from concept through design to full implementation [3]. It equips the engineer with methods and tools to handle systems of ever-increasing complexity arising from greater interactions within these systems or from the expanding heterogeneity they demonstrate in their structure and function. Despite its many accomplishments, model-based systems engineering still relies on graphical modeling languages that provide limited quantitative insight (on their own) [4–6].

In contrast, network science has developed to quantitatively analyze networks that appear in a wide variety of engineering systems [7]. Network science focuses on an abstract model of a system's form, neglecting an explicit description of a system's function [8, 9]. It utilizes formal graphs, comprising nodes and edges [8, 9]. Nodes typically represent facilities associated with point locations, while edges represent facilities that connect the nodes. The nodes and edges in a formal graph are described by nouns. Because many engineering systems include multiple elements with several layers of connectivity, formal graphs are frequently generalized to create multilayer networks. In either case, operands are transported along edges between pairs of nodes, whether in a single layer or multiple layers. And yet, despite its methodological developments in multilayer networks, network science has often been unable to address the explicit heterogeneity frequently encountered in engineering systems [6, 10]. In a comprehensive review, Kivela et al. [10] write: "[The multi-layer network community] has produced an equally immense explosion of disparate terminology, and the lack of consensus (or even generally accepted) set of terminology and mathematical framework for studying is extremely problematic."

1.1 | Original Contribution

This work provides a conceptual introduction to HFGT. Based on prior works, it serves as an entry point to HFGT for MBSE and network science audiences. In many ways, the parallel developments of the model-based systems engineering and network science communities converge intellectually in HFGT [6]. For the latter, it utilizes multiple graph-based data structures to support a matrix-based quantitative analysis. For the former, HFGT inherits the heterogeneity of conceptual and ontological constructs found in model-based systems engineering, including system form, system function, and system concept. More specifically, the explicit treatment of function and operand facilitates a structural understanding of the diversity of engineering systems found in Table 1. In effect, HFGT quantifies many of the structural

TABLE 1 | A classification of engineering systems by function and operand [2].

	Operands				
Function	**Living organisms**	**Matter**	**Energy**	**Information**	**Money**
Transform	Hospital	Blast furnace	Engine, electric motor	Analytic engine, calculator	Bureau of printing and engraving
Transport	Car, airplane, train	Truck, train, car, airplane	Electricity grid	Cables, radio, telephone, and internet	Banking fedwire and SWIFT transfer systems
Store	Farm, apartment complex	Warehouse	Battery, flywheel, capacitor	Magnetic tape and disk, book	U.S. bullion repository (Fort knox)
Exchange	Cattle auction, (illegal) human trafficking	eBay trading system	Energy market	World wide web, Wikipedia	London stock exchange
Control	U.S. constitution and laws	National highway traffic safety administration	Nuclear regulatory commission	Internet engineering task force	U.S. federal reserve

concepts found in model-based systems engineering and its associated languages (e.g., UML/SysML). Although not named as such initially, the first works on HFGT appeared as early as 2006-2008 [11–14]. Since then, HFGT has become multiply established and demonstrated cross-domain applicability [6, 15]; culminating in several recent consolidating works [1, 6, 16]. This work captures the most essential aspects of these works to give MBSE and network science audiences a straightforward path to adopting HFGT in their applications.

The ontological strength of HFGT comes from the "systems thinking" foundations in the model-based systems engineering literature [6, 9]. In effect, and very briefly, all systems have a "subject + verb + operand" form where the system form is the subject, the system function is the verb + operand (i.e., predicate) and the system concept is the mapping of the two to each other. The key distinguishing feature of HFGT, relative to multilayer networks, is its introduction of system function. In that regard, it is more complete than multilayer networks if the system function is accepted as part of an engineering system abstraction. Another key distinguishing feature of HFGT is the differentiation between elements related to transformation, transportation, and storage. In that regard, it takes great care not to *overload* mathematical modeling elements and preserve lucidity. These diverse conceptual constructs indicate multi-dimensional rather than two-dimensional relationships. Finally, HFGT has been shown to be capable of modeling an arbitrary number of engineering systems of arbitrary topology, connected to each other arbitrarily, within a system-of-systems. As Section 5 elaborates, these modeling and analytical strengths have been utilized to address a range of analytical challenges in various application domains.

1.2 | Paper Outline

The remainder of the article is organized as follows. Section 2 describes the ontological underpinnings of HFGT. It explains why an ontological analysis is so critical and how HFGT preserves ontologies as they are translated from conceptual mental models to shared graphical models and mathematical models. Section 3 then discusses the use of meta-architectures as a means of overcoming the convergence challenges presented by domain-specific reference architectures and case-specific instantiated architectures. More specifically, the HFGT meta-architecture is advanced as a means of overcoming the ontological limitations found in several meta-architectures advanced in the literature. Next, Section 4 describes some of the essential elements of HFGT. These elements provide the MBSE and network science audience with a sufficient introduction on how to construct hetero-functional graphs in terms of their incidence and adjacency matrices. Equally important, the section relates hetero-functional graphs to the perhaps more familiar formal graphs and multilayer networks. Finally, Section 5 provides an entry point to further reading on HFGT, thus providing an opportunity to build upon this conceptual introduction with practical applications.

2 | The Role of Systems Ontology

HFGT draws on ontological science and can be used to compare the modeling fidelity of HFGT to the multilayer network literature. For example, it has been shown that HFGT overcomes eight previously identified modeling constraints in multilayer networks [17, 18]. The modeling constraints found in multilayer networks can be viewed from an ontological perspective. As shown in Figure 1, ontological science describes the relationship between reality (the Real Domain), the understanding of reality (the Domain Conceptualization), and the description of reality (the Language). These general concepts are instantiated to describe the modeling of a Physical System, where the modeler's understanding (or mental conceptualization) is the Abstraction, and the description of the abstraction is the Model.

The modeling process abstracts the physical system into an abstraction and represents the abstraction with a model. The model refers to the physical system, but this reference is always indirect, as an abstraction is always made in the modeling process. The abstraction of reality may be entirely conceptual (residing within the mind) or linguistic (residing within

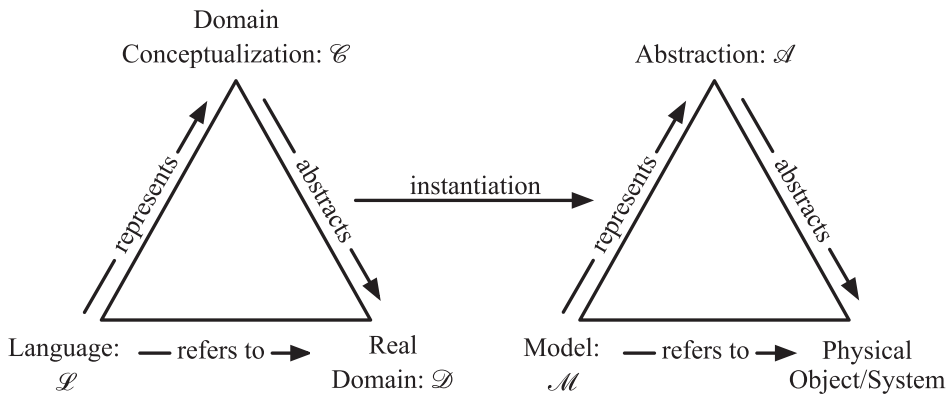

FIGURE 1 | Ullman's triangle [19]: Its ontological definition. On the left, the relationship between reality, the understanding of reality, and the description of reality. On the right, the instantiated version of the definition.

some predefined language). For a model to truly represent the abstraction, the modeling primitives of the language should faithfully represent the domain conceptualization to articulate the represented abstraction. As a result, modeling primitives directly express relevant domain concepts, creating the domain conceptualization. The fidelity of the model with respect to the abstraction is determined by four complementary linguistic properties: soundness, completeness, lucidity, and laconicity. When all four properties are met, the abstraction and the model have a one-to-one mapping and faithfully represent each other.

Definition 2 Soundness [20]. A language is sound with respect to a domain conceptualization if every modeling primitive in the language has an interpretation in the domain abstraction (the absence of soundness results in excess modeling primitives with respect to domain abstractions).

Definition 3 Completeness [20]. A language is complete with respect to a domain conceptualization if every concept in the domain abstraction of that domain is represented in a modeling primitive of the language (the absence of completeness results in one or more concepts in the domain abstraction not being represented by a modeling primitive).

Definition 4 Lucidity [20]. A language is lucid with respect to a domain conceptualization if every modeling primitive in the language represents at most one domain concept in the domain abstraction (the absence of lucidity results in the overload of a modeling primitive with respect to two or more domain concepts).

Definition 5 Laconicity [20]. A language is laconic with respect to a domain conceptualization if every concept in the abstraction of that domain is represented at most once in the model of that language (the absence of laconicity results in the redundancy of modeling primitives with respect to domain abstractions)

It is essential to distinguish between situations where the abstraction is entirely conceptual, residing within the mind, and those where it is linguistic, residing within a predefined language. In the case of the former, as is practiced in model-based systems engineering, the abstraction of the system is an instance of "systems thinking" as the domain conceptualization. Then, the (SysML) model of the system is an instance of the Systems Modeling Language (SysML) as the language. This is the default for many small- to medium-scale engineering systems. In the case of the latter, as is practiced in HFGT, the abstraction is the SysML model, and the domain conceptualization is SysML. The hetero-functional graph model(s) are the model, and HFGT provides the means of representing the domain conceptualization in a mathematical language. This is the case when the engineering system is so large and complex (e.g., a system-of-systems) that a single human mind cannot retain the entire domain conceptualization. Regardless of whether the abstraction is conceptual or linguistic, ontological science demands one-to-one mappings of the concepts in the mind, their graphical counterparts in SysML, and finally their mathematical counterparts in HFGT.

3 | Instantiated, Reference, and Meta-Architectures for Systems-of-Systems

Ullman's triangle [19] in Figure 1 provides a basis for understanding the convergence challenge found in complex systems-of-systems. Consider what happens when the physical/object system is now a system-of-systems. Also, assume that the system-of-systems is a system of "unlike" systems that have heterogeneity of type between the constituent systems. Consequently, the real domain is a real domain of real domains that unites the constituent domains into one. For example, the study of the food-energy-water nexus in the general (rather than a specific region) may constitute such a real-domain-of-real-domains. It needs to be abstracted by human beings, in the mind, in a domain-conceptualization of domain-conceptualizations. Similarly, that must be referred to by a language of languages. Several convergence challenges immediately arise. First, human beings are typically trained in a single-domain conceptualization, rather than multiple domains. Indeed, it is far from clear that there even exists a single human being (let alone many) who has sufficient knowledge of the domain-conceptualization-of-domain-conceptualizations. In the absence of such an individual, a group of individuals—each with their own domain conceptualizations—must somehow collaborate to discuss the

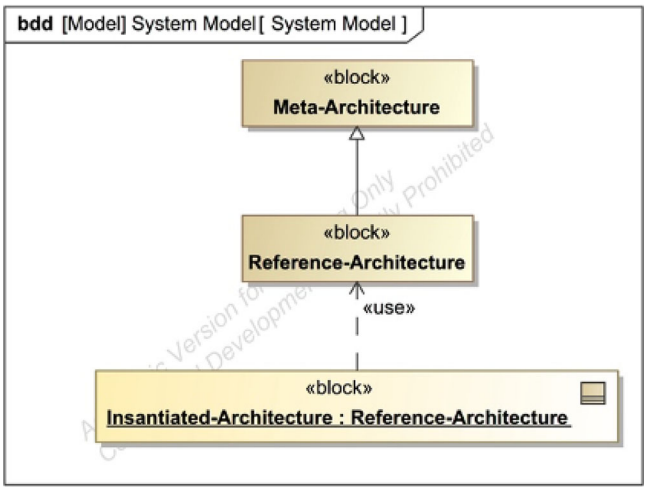

FIGURE 2 | SysML block definition diagram. Systems architecture can be represented at three levels of abstraction: instantiated, reference, and meta-architecture [17].

real-domain-of-real-domains (e.g., the food-energy-water nexus independent of a specific region). They immediately find that each domain-conceptualization comes with its associated language, and a language of languages emerges. Because each of these languages was developed entirely independently to address the needs of its associated real domain, the language-of-languages is highly divergent, and a common, convergent understanding between languages is difficult to achieve. To overcome this impasse, the language-of-language may develop a translation capability between each of the languages about each real domain. While this strategy is relatively straightforward for only two languages with a single translator, it does not scale when there are N real-domains that require potentially N(N-1) translators between N languages. The remaining alternative is to invest in the development of a language-of-language that reconciles the languages together into a single common language.

MBSE, SysML, and HFGT adopt the latter approach where a single common language serves as a language-of-languages [6, 21]. The development of a single common language for a domain-conceptualization-of-domain-conceptualizations requires three types of system architectures. As shown in Figure 2, these are the instantiated, reference, and meta-architectures. Each of these is explained in turn.

Generally speaking, a system architecture consists of three parts: the physical architecture, the functional architecture, and the mapping of the latter onto the former in a system concept (or allocated architecture) [9, 21]. The physical architecture is a description of the decomposed elements of the system, without specifying the performance characteristics of the physical resources that comprise each element. The functional architecture is a description of the system processes in a solution-neutral way, structured in serial, parallel, or hierarchical arrangements. The system concept, as a mapping of the functional architecture onto the physical architecture, completes the system architecture. To respect the Independence Axiom [22], and for a hetero-functional graph model to be correctly specified, it is assumed that the entirety of any given process must be completed by a given resource. While this assumption may seem limiting, in reality, either the process or the resource can be decomposed so that the Independence Axiom is ultimately respected.

An Instantiated Systems Architecture is a case-specific architecture that represents a real-world scenario [6]. At this level, the physical architecture comprises a set of instantiated resources, and the functional architecture comprises a set of instantiated system processes. The mapping in the system concept defines which resources perform what processes.

Reference Architectures generalize instantiated system architecture [6, 23]. Instead of using individual instances as elements of the physical and functional architecture, the reference architecture is expressed in terms of domain-specific classes of these instances. In this way, the reference architecture captures the essence of existing instantiated architectures. It also provides a vision of future needs that can guide the development of new instantiated system architectures. Such a reference architecture facilitates a shared understanding across multiple disciplines or organizations about the current architecture and its future evolution. A reference architecture is based on concepts that have been proven in practice. Most often, preceding architectures are mined for these proven concepts. The reference architecture, therefore, generalizes instantiated system architectures to define an architecture that is generally applicable in a discipline. However, the reference architecture does not generalize beyond the domain conceptualization.

Meta-architectures further generalize reference architectures [6]. Instead of domain-specific elements, it is expressed in terms of domain-neutral classes. A meta-architecture is composed of "primitive elements" that generalize the domain-specific functional and physical elements into their domain-neutral equivalents [24]. While no single engineering system meta-architecture has been developed for all purposes, several modeling methodologies have been developed that span several discipline-specific domains. In the design of dynamic systems, bond graphs [25–27] and linear graphs [28–32] use generalized capacitors, resistors, inductors, gyrators, and transformers as primitive elements. In the system dynamics of business, stocks and flows are often used as primitives [33, 34]. Finally, formal graph theory [35, 36] introduces nodes and edges as primitive elements. Each of these has its respective set of applications. However, their sufficiency must ultimately be tested by an ontological analysis of soundness, completeness, lucidity, and laconicity. HFGT, as the next section elaborates, utilizes its own meta-architecture, which, in recent years, has been shown to generalize linear graphs, bond graphs, system dynamics, and formal graph theory [1, 37, 38]. Given the importance of ontological clarity, HFGT has taken special care in translating this meta-architecture from its description in SysML to its mathematical representation.

4 | Essential Elements of HFGT

While a comprehensive introduction to HFGT cannot be provided here, this section introduces the essential elements of HFGT in terms of its underlying meta-architecture. Unlike other meta-architectures, HFGT is rooted in the universal structure of human language, which features subjects and predicates, with the latter

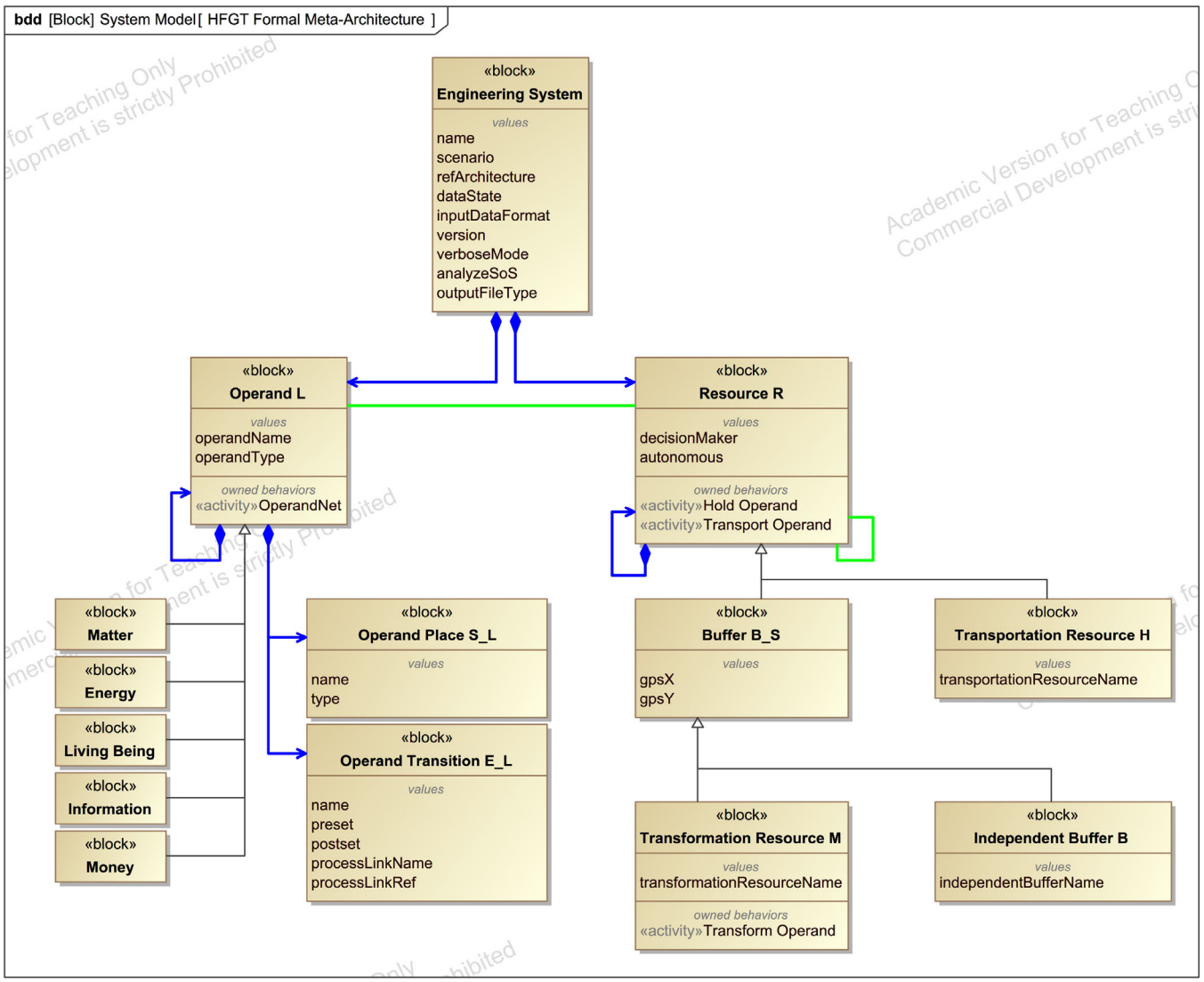

FIGURE 3 | A SysML block diagram of the HFGT formal meta-architecture [6].

comprising verbs and objects [1, 6]. An engineering system includes a set of resources R as subjects, a set of system processes P as predicates, and a set of operands L as their constituent objects.

Definition 6 System Operand [3]. An asset or object $l_i \in L$ that is operated on or consumed during the execution of a process.

Definition 7 System Process [3, 39]. An activity $p \in P$ that transforms a predefined set of input operands into a predefined set of outputs.

Definition 8 System Resource [3]. An asset or object $r_v \in R$ that is utilized during the execution of a process.

As shown in Figure 3, these operands, processes, and resources are organized in an engineering system meta-architecture stated in the Systems Modeling Language [4, 21, 40]. As in SysML, the processes can be discrete or streaming, and no limiting assumption is made on their temporal or spatial scale. Importantly, operands in the engineering system have several types, including matter, energy, living beings, information, and money, to correspond to those operands found in Table 1. Interestingly, it is understood that operands, in general, have some state in time. The evolution of this operand state is described by an operand net as a type of Petri net [41].

Definition 9 Operand Net [6, 14, 42, 43]. Given operand l_i, an elementary Petri net $N_{l_i} = \{S_{l_i}, E_{l_i}, M_{l_i}, Q_{l_i}\}$ where:

- S_{l_i} is the set of places describing the operand's state.
- E_{l_i} is the set of transitions describing the evolution of the operand's state.
- M_{l_i} is the set of weighted arcs from places to transitions and transitions back to places, with the associated incidence matrices:

$$M_{l_i} = M_{l_i}^{+} - M_{l_i}^{-}.$$

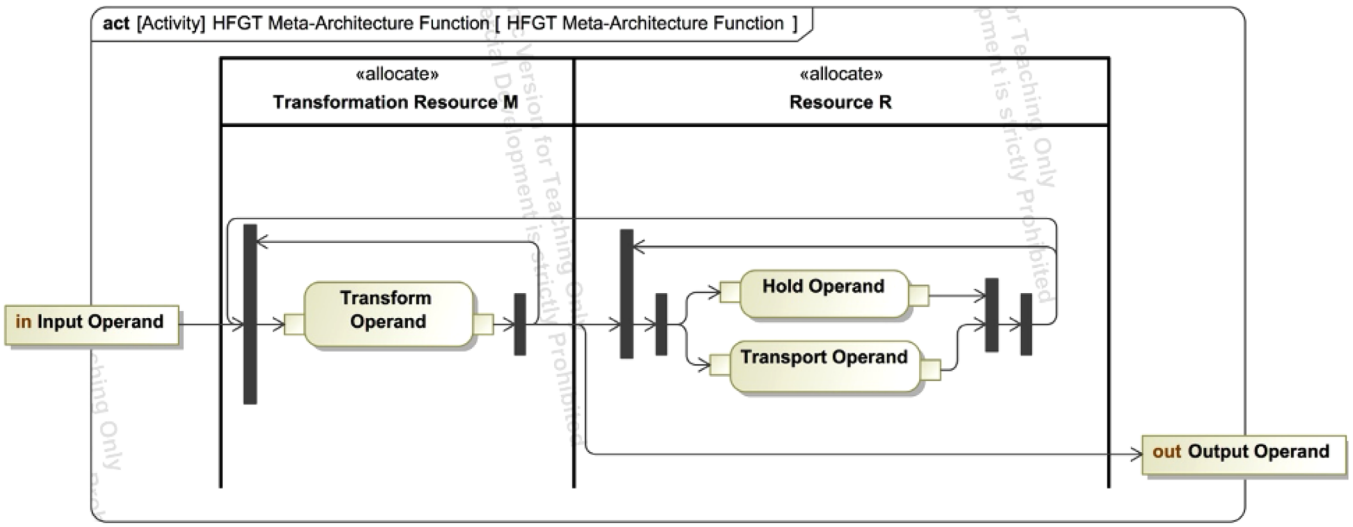

FIGURE 4 | A SysML activity diagram of the HFGT functional meta-architecture [6].

The middle of Figure 3 shows that the system resources $R = M \cup B \cup H$ are classified into transformation resources M, independent buffers B, and transportation resources H.

Definition 10 Buffer [1, 6]. A resource $r_y \in R$ is a buffer $b_s \in B_s$ if and only if it is capable of storing or transforming one or more operands at a unique location in space. $B_s = M \cup B$.

Definition 11 Capability [1, 6, 24]. : An action $e_{wv} \in \mathcal{E}_S$ (in the SysML sense) is defined by a system process $p_w \in P$ being executed by a resource $r_v \in R$. It constitutes a subject + verb + operand sentence of the form: *Resource* r_v does process p_w.

Additionally, the set of buffers $B_s = M \cup B$ is introduced as an important part of the theory. HFGT inherently supports cyber-physical and socio-technical systems. The transformation resources, independent buffers, and transportation resources can be physical in nature because they act upon matter, energy, and living beings. Alternatively, they can be decision-making resources that act upon information and money. Finally, Figure 3 shows that resources (and their associated types) are capable of executing one or more system processes to produce a set of capabilities [6].

The system processes $P = P_\mu \cup P_\eta$ are classified into transformation processes P_μ and refined transportation processes P_η [6]. The latter arises from the simultaneous execution of one (unrefined) transportation process and one holding process. The relationships between the different types of system processes and the resources that can execute them are further depicted in the HFGT functional meta-architecture (Figure 4).

Figures 3 and 4 both show that only transformation resources can carry out transformation processes, while all resources can carry out refined transportation processes. Figure 4 also depicts the simultaneous execution of an (unrefined) transportation process and holding process in the place of a refined transportation process. Finally, Figure 4 shows that transformation processes and refined transportation processes can follow each other in any arbitrary sequence.

Returning to Defn. 11, it is essential to recognize that while capabilities are their own distinct entities, in reality, they are formed by the allocation of a process p_w to a resource r_v. In Figure 3, these capabilities appear as owned behaviors of their respective blocks. In Figure 4, these capabilities appear as actions in their respective swim lanes. At an engineering system level, these allocations are described in the system concept.

Definition 12 System Concept [12, 15, 44]. A binary matrix A_S of size $|P| \times |R|$ whose element $A_S(w, v) \in \{0, 1\}$ is equal to one when action $e_{wv} \in \mathcal{E}_S$ (in the SysML sense) is available as a system process $p_w \in P$ being executed by a resource $r_v \in R$.

In other words, the system concept forms a bipartite graph between the set of system processes and the set of system resources [15].

Once the engineering system's capabilities have been defined, it is necessary to understand how they interact with one another. These appear most clearly as the directed arrows between the actions allocated to swim lanes in Figure 4. Mathematically, HFGT describes these functional interactions with the (third-order) hetero-functional incidence tensor $\mathcal{M}_\rho = \mathcal{M}_\rho^+ - \mathcal{M}_\rho^-$.

Definition 13 The Negative 3rd Order Hetero-functional Incidence Tensor $\widetilde{\mathcal{M}}_\rho^-$ [1]. The negative hetero-functional incidence tensor $\widetilde{\mathcal{M}}_\rho^- \in \{0, 1\}^{|L| \times |B_S| \times |\mathcal{E}_S|}$ is a third-order tensor whose element $\widetilde{\mathcal{M}}_\rho^-(i, y, \psi) = 1$ when the system capability $\epsilon_\psi \in \mathcal{E}_S$ pulls operand $l_i \in L$ from buffer $b_{s_y} \in B_S$.

Definition 14 The Positive 3rd Order Hetero-functional Incidence Tensor $\widetilde{\mathcal{M}}_\rho^+$ [1]. The positive hetero-functional incidence tensor $\widetilde{\mathcal{M}}_\rho^+ \in \{0, 1\}^{|L| \times |B_S| \times |\mathcal{E}_S|}$ is a third-order tensor whose element $\widetilde{\mathcal{M}}_\rho^+(i, y, \psi) = 1$ when the system capability $\epsilon_\psi \in \mathcal{E}_S$ injects operand $l_i \in L$ into buffer $b_{s_y} \in B_S$.

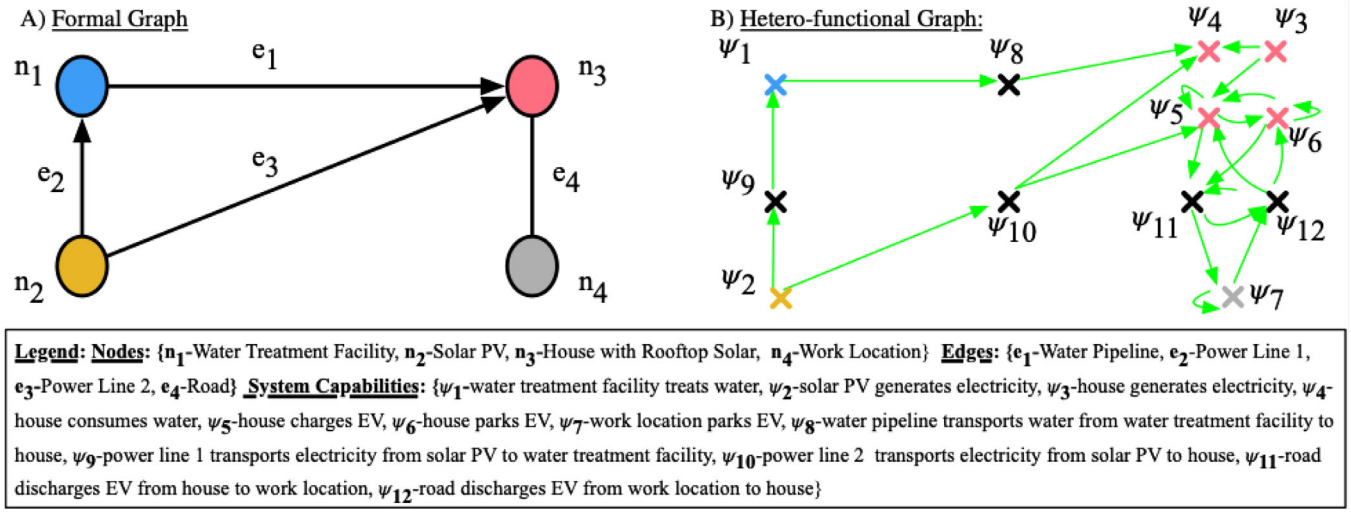

FIGURE 5 | A visual comparison of a formal graph (FG) and a hetero-functional graph (HFG) model of the same hypothetical system. [6].

Definition 15 (Hetero-functional Adjacency Matrix (Projected)). [15, 43, 45–47] A square binary matrix A_ρ of size $|E_S| \times |E_S|$ whose element $A_\rho(\psi_1, \psi_2) \in \{0, 1\}$ is equal to one when string $z_{\{\psi_1,\psi_2\}} \in Z$ is available and exists.

Interestingly, the positive and negative third-order hetero-functional incidence tensors have closed-form formulas that make their automated calculation straightforward [1]. Furthermore, these third-order incidence tensors can be straightforwardly matricized to form their associated second-order hetero-functional incidence matrices $M_\rho = M_\rho^+ - M_\rho^-$ with dimensions $|L||B_S| \times |E_S|$. This enables the formation of a hetero-functional graph and its corresponding adjacency matrix.

Interestingly, the (projected) hetero-functional adjacency matrix is straightforwardly calculated from either the second-order or third-order positive and negative hetero-functional incidence tensors [1, 48].

$$A_\rho(\psi_1, \psi_2) = \sum_{i}^{|L|} \sum_{y}^{|B_S|} M_\rho^+(i, y, \psi_1) \cdot M_\rho^-(i, y, \psi_2) \tag{1}$$

$$A_\rho = M_\rho^{+T} M_\rho^- \tag{2}$$

This highly abridged conceptual introduction to HFGT can also be explained graphically through the illustrative example shown in Figure 5. It illustrates the difference between a formal graph and a hetero-functional graph (HFG). The formal graph in Figure 5a shows a system composed of four nodes: a water treatment facility, a solar PV panel, a house with rooftop solar, and a work location. These are connected by four edges: a water pipeline, two power lines, and two roads. In contrast, Figure 5b shows the associated hetero-functional graph. Instead of four nodes that represent point-like facilities, the hetero-functional graph now has 12 nodes that represent the connected system capabilities. The water treatment facility, solar PV panel, and work location appear unchanged between the two graphs because they each have only one capability. In contrast, the house with rooftop solar provides four capabilities in the HFG. Thirdly, the edges in the formal graph are now transportation capabilities connecting nodes in the HFG. Finally, the directed edges in the HFG indicate the logical sequences of these capabilities such that if one were to follow them, a "story" of capabilities would emerge. (i.e., The water treatment facility treats water (ψ_1) and then the water pipeline transports the water from the water treatment facility to the house (ψ_8)). This example is further discussed and elaborated extensively in the HFGT textbook [6].

Interestingly, the adjacency matrix pertaining to the formal graph can also be calculated from the third-order positive and negative hetero-functional incidence tensors [1, 48].

$$A_{B_S}(y_1, y_2) = \bigvee_{\psi}^{|\mathcal{E}_S|} \bigvee_{i}^{|L|} M_\rho^-(i, y_1, \psi) \cdot M_\rho^+(i, y_2, \psi) \tag{3}$$

It has a similar mathematical form to Equation 1 but collapses the dimension pertaining to the system capabilities rather than collapsing the dimension pertaining to the system buffers. Additionally, the adjacency tensor pertaining to a multilayer formal graph can also be calculated from the third-order positive and negative hetero-functional incidence tensors [1, 48].

$$A_{B_S}(y_1, y_2, i_1, i_2) = \bigvee_{\psi}^{|\mathcal{E}_S|} M_\rho^-(i_1, y_1, \psi) \cdot M_\rho^+(i_1, y_2, \psi) \tag{4}$$

This collapsing of dimensions in formal graphs and their multilayer networks is the reason why both have been proven to lack ontological lucidity and completeness [1]. Hetero-functional graphs, on the other hand, have no such ontological limitation because a given capability $e_p si$ decomposes to its associated resource r_v and process p_w, which in turn imply the associated operands L and buffers B_S.

5 | Further Reading on HFGT

In summary, HFGT [1, 6, 24] has emerged as a tool for studying many complex engineering systems and systems-of-systems. More specifically, HFGT provides a means of algorithmically

translating SysML models [4, 21, 40] into hetero-functional graphs and/or Petri Nets [41]. Methodologically speaking, in comparison to the ontological and modeling limitations of multilayer networks [10], HFGT has demonstrated its ability to model an arbitrary number of networked systems of arbitrary topology connected arbitrarily [6]. Furthermore, it has demonstrated its ability to reconcile and subsequently generalize both multilayer networks [1, 48], axiomatic design models [24, 49], system dynamics models [38], bond graph models [37], and linear graph models [37]. The direct relationship between these disparate modeling and analysis techniques facilitates the cross-pollination of theoretical and practical results between these approaches. For example, bond graphs are often used to study system causality [26], where such an analysis has been elusive in linear graphs, multilayer networks, and hetero-functional graphs. This ability to reconcile system models from disparate disciplinary sources, as well as the ability to model systems-of-systems of arbitrary topology, allows HFGT to conduct novel structural analyses [15, 50–52], dynamic simulations, [53], and optimal decision problems [16]. In all three of these categories, there remain ample opportunities for methodological contributions.

HFGT has also been applied to numerous application domains individually, including electric power [45, 50], potable water [15], transportation [46], and mass-customized production systems [13, 14, 54, 55]. Perhaps more importantly, HFGT has already demonstrated its applicability to systems-of-systems including multi-modal electrified transportation systems [47, 56, 57], microgrid-enabled production systems [43], personalized healthcare delivery systems [42, 53, 58], hydrogen-natural gas systems [16], energy-water-nexus [52], engineering management of mega-projects [59, 60], and the American multi-modal energy system [51]. Interestingly, several of these systems-of-systems applications of HFGT combine both time-driven and discrete-event phenomena. Additionally, HFGT has demonstrated its ability to model the supply, demand, transportation, storage, transformation, assembly, and disassembly of multiple operands in distinct locations across multiple systems over time [16]. Finally, these multiple operands include matter, energy, information, money, and living organisms [1, 6], and not just energy, as in engineering-centric methods such as linear graphs and bond graphs. At this stage of its development, HFGT has overcome many limitations in these disparate application domains and is likely to continue to do so. Meanwhile, its generic mathematical approach and its ties to MBSE and SysML suggest that it will continue to find utility in additional application domains that have yet to be investigated.

Acknowledgments

This research is based on work supported by the Growing Convergence Research Program of the National Science Foundation under Grant Numbers OIA-2317874 and OIA-2317877.

Data Availability Statement

Data sharing not applicable to this article as no datasets were generated or analysed during the current study.

References

1. A. M. Farid, D. Thompson, and W. C. Schoonenberg, "A Tensor-Based Formulation of Hetero-Functional Graph Theory," *Nature Scientific Reports* 12, no. 18805 (2022): 1–22, https://doi.org/10.1038/s41598-022-19333-y.

2. O. L. De Weck, D. Roos, and C. L. Magee, *Engineering Systems: Meeting Human Needs in a Complex Technological World* (MIT Press, 2011), http://www.knovel.com/knovel2/Toc.jsp?BookID=4611http://mitpress-ebooks.mit.edu/product/engineering-systems.

3. SE Handbook Working Group, *Systems Engineering Handbook: A Guide for System Life Cycle Processes and Activities* (International Council on Systems Engineering (INCOSE), 2015).

4. T. Weilkiens, *Systems Engineering With SysML/UML Modeling, Analysis, Design* (Morgan Kaufmann, 2007).

5. S. Friedenthal, A. Moore, and R. Steiner, *A Practical Guide to SysML: The Systems Modeling Language*, 2nd ed. (Morgan Kaufmann, 2011).

6. W. C. Schoonenberg, I. S. Khayal, and A. M. Farid, *A Hetero-functional Graph Theory for Modeling Interdependent Smart City Infrastructure* (Springer, 2019), https://doi.org/10.1007/978-3-319-99301-0.

7. W. K. Harrison, "The Role of Graph Theory in System of Systems Engineering," *IEEE Access* 4 (2016): 1716–1742.

8. M. Pósfai and A.-L. Barabási, "Network Science," in *posfai2016network* 3 (Cambridge University Press, 2016), https://tinyurl.com/4nzdbed6.

9. E. Crawley, B. Cameron, and D. Selva, *System Architecture: Strategy and Product Development for Complex Systems* (Prentice Hall Press, 2015).

10. M. Kivelä, A. Arenas, M. Barthelemy, J. P. Gleeson, Y. Moreno, and M. A. Porter, "Multilayer Networks," *Journal of Complex Networks* 2, no. 3 (2014): 203–271.

11. A. M. Farid and D. C. McFarlane, "A Development of Degrees of Freedom for Manufacturing Systems," in *IMS'2006: 5th International Symposium on Intelligent Manufacturing Systems: Agents and Virtual Worlds* (Sakarya, Turkey, 2006), 1–6, http://liines.net/resources/Conferences/IEM-C02.pdf.

12. A. M. Farid, "Reconfigurability Measurement in Automated Manufacturing Systems," (Ph.D. Dissertation, University of Cambridge Engineering Department Institute for Manufacturing, 2007), http://liines.net/resources/Theses/IEM-TP00.pdf.

13. A. M. Farid and D. C. McFarlane, "Production Degrees of Freedom as Manufacturing System Reconfiguration Potential Measures," in *Proceedings of the Institution of Mechanical Engineers, Part B (Journal of Engineering Manufacture) – Invited Paper* 222, no. B10 (2008): 1301–1314, https://doi.org/10.1243/09544054JEM1056.

14. A. M. Farid, "Product Degrees of Freedom as Manufacturing System Reconfiguration Potential Measures," *International Transactions on Systems Science and Applications – Invited Paper* 4, no. 3 (2008): 227–242, http://liines.net/resources/Journals/IEM-J04.pdf.

15. A. M. Farid, "Static Resilience of Large Flexible Engineering Systems: Axiomatic Design Model and Measures," *IEEE Systems Journal* no. 99 (2015): 1–12, https://doi.org/10.1109/JSYST.2015.2428284.

16. W. C. Schoonenberg and A. M. Farid, "Hetero-Functional Network Minimum Cost Flow Optimization," *Sustainable Energy Grids and Networks* 31, no. 100749 (2022): 1–18, https://doi.org/10.1016/j.segan.2022.100749.

17. W. C. Schoonenberg, I. S. Khayal, and A. M. Farid, *A Hetero-Functional Graph Theory for Modeling Interdependent Smart City Infrastructure* (Springer, 2019).

18. M. De Domenico, A. Solé-Ribalta, E. Cozzo, M. Kivelä, Y. Moreno, M. A. Porter, S. Gómez, and A. Arenas, "Mathematical Formulation of Multilayer Networks," *Physical Review X* 3, no. 4 (2013): 041022.

19. G. Guizzardi, "On Ontology, Ontologies, Conceptualizations, Modeling Languages, and (Meta) Models," *Frontiers in Artificial Intelligence and Applications* 155 (2007): 18.

20. G. Guizzardi, "Ontological Foundations for Structural Conceptual Models," (Ph.D. Dissertation, University of Twente, 2005).

21. L. Delligatti, *SysML Distilled: A Brief Guide to the Systems Modeling Language* (Addison-Wesley, 2014).

22. N. P. Suh, *Axiomatic Design: Advances and Applications* (Oxford University Press, 2001).

23. R. Cloutier, G. Muller, D. Verma, R. Nilchiani, E. Hole, and M. Bone, "The Concept of Reference Architectures," *Systems Engineering* 13, no. 1 (2010): 14–27.

24. A. M. Farid, "An Engineering Systems Introduction to Axiomatic Design," in *Axiomatic Design in Large Systems: Complex Products, Buildings & Manufacturing Systems* ed., A. M. Farid and N. P. Suh (Heidelberg: Springer, 2016), ch. 1, 1–47, https://doi.org/10.1007/978-3-319-32388-6.

25. H. M. Paynter, *Analysis and Design of Engineering Systems* (MIT press, 1961).

26. D. Karnopp, D. L. Margolis, and R. C. Rosenberg, *System Dynamics: A Unified Approach*, 2nd ed. (Wiley, 1990), http://www.loc.gov/catdir/enhancements/fy0650/90012110-t.html.

27. F. T. Brown, *Engineering System Dynamics*, 2nd ed. (CRC Press Taylor & Francis Group, 2007).

28. H. E. Koenig, Y. TOKAD, and H. K. Kesavan, *Analysis of Discrete Physical Systems* (McGraw-Hill, 1967).

29. W. A. Blackwell, *Mathematical Modeling of Physical Networks* (Collier-Macmillan, 1968).

30. J. L. Shearer, A. T. Murphy, and H. H. Richardson, *Introduction to System Dynamics*, vol 7017 (Addison-Wesley Reading, 1967).

31. B. C. Kuo, *Linear Networks and Systems* (McGraw-Hill, 1967).

32. S.-P. Chan, S.-Y. Chan, S.-G. Chan, et al., *Analysis of Linear Networks and Systems* (Mass. Addison-Wesley, 1972).

33. J. W. Forrester, "Industrial Dynamics: A Major Breakthrough for Decision Makers," *Harvard Business Review* 36, no. 4 (1958): 37–66.

34. J. D. Sterman, *Business Dynamics: Systems Thinking and Modeling for a Complex World*, vol 19 (Irwin/McGraw-Hill Boston, 2000).

35. M. van Steen, *Graph Theory and Complex Networks: An Introduction* (Maarten van Steen, 2010).

36. M. Newman, *Networks: An Introduction* (Oxford University Press, 2009), http://books.google.ae/books?id=LrFaU4XCsUoC.

37. E. Ghorbanichemazkati and A. M. Farid, "Generalizing Linear Graphs and Bond Graph Models With Hetero-Functional Graphs for System-of-Systems Engineering Applications," preprint, arXiv, September 5, 2024, https://arxiv.org/abs/2409.03630.

38. M. Naderi, M. Harris, J. C. Little, and A. M. Farid, "Simulating Mono Lake System: Convergence Paradigm by the Hetero-Functional Graph Theory Approach or a System Dynamics Model?" in *1st Workshop of Systems of Systems Applications of Hetero-functional Graph Theory* (Annapolis, MA, USA, 2024).

39. D. Hoyle, *ISO 9000 Pocket Guide* (Butterworth-Heinemann, 1998), http://www.loc.gov/catdir/toc/els033/99163006.html.

40. S. Friedenthal, A. Moore, and R. Steiner, *A Practical Guide to SysML: The Systems Modeling Language* (Morgan Kaufmann, 2014).

41. C. Girault and R. Valk, *Petri Nets for Systems Engineering: A Guide to Modeling, Verification, and Applications* (Springer Science & Business Media, 2013).

42. I. S. Khayal and A. M. Farid, "Architecting a System Model for Personalized Healthcare Delivery and Managed Individual Health Outcomes," *Complexity* 1, no. 1 (2018): 1–25, https://doi.org/10.1155/2018/8457231.

43. W. C. Schoonenberg and A. M. Farid, "A Dynamic Model for the Energy Management of Microgrid-Enabled Production Systems," *Journal of Cleaner Production* 1, no. 1 (2017): 1–10, https://dx.doi.org/10.1016/j.jclepro.2017.06.119.

44. A. M. Farid and N. P. Suh, eds., *Axiomatic Design in Large Systems: Complex Products, Buildings and Manufacturing Systems* (Springer, 2016), https://doi.org/10.1007/978-3-319-32388-6.

45. A. M. Farid, "Multi-Agent System Design Principles for Resilient Coordination and Control of Future Power Systems," *Intelligent Industrial Systems* 1, no. 3 (2015): 255–269, https://doi.org/10.1007/s40903-015-0013-x.

46. A. Viswanath, E. E. S. Baca, and A. M. Farid, "An Axiomatic Design Approach to Passenger Itinerary Enumeration in Reconfigurable Transportation Systems," *IEEE Transactions on Intelligent Transportation Systems* 15, no. 3 (2014): 915–924, https://doi.org/10.1109/TITS.2013.2293340.

47. A. M. Farid, "A Hybrid Dynamic System Model for Multi-Modal Transportation Electrification," *IEEE Transactions on Control System Technology* no. 99 (2016): 1–12, https://doi.org/10.1109/TCST.2016.2579602.

48. D. Thompson and A. M. Farid, "Reconciling Formal, Multi-layer, and Hetero-Functional Graphs With the Hetero-Functional Incidence Tensor," in *IEEE Systems of Systems Engineering Conference* (Rochester, 2022), 1–6, https://doi.org/10.1109/SOSE55472.2022.9812692.

49. G.-J. Park and A. M. Farid, "Design of Large Engineering Systems," in *Design Engineering and Science*, N. P. Suh, ed. M. Cavique, and J. Foley (Springer, 2021), 367–415, https://doi.org/10.1007/978-3-030-49232-8_14.

50. D. Thompson, W. C. Schoonenberg, and A. M. Farid, "A Hetero-Functional Graph Resilience Analysis of the Future American Electric Power System," *IEEE Access* 9 (2021): 68837–68848, https://doi.org/10.1109/ACCESS.2021.3077856.

51. D. Thompson and A. M. Farid, "A Hetero-Functional Graph Structural Analysis of the American Multi-Modal Energy System," *Sustainable Energy, Grids, and Networks* 38, no. 1 (2024): 101254–101269, https://doi.org/10.1016/j.segan.2023.101254.

52. A. M. Farid, "A Hetero-Functional Graph Resilience Analysis for Convergent Systems-of-Systems," *International Journal of Systems Science* (2025): 1–26, https://doi-org/10.1080/00207721.2025.2602891.

53. I. S. Khayal and A. M. Farid, "A Dynamic System Model for Personalized Healthcare Delivery and Managed Individual Health Outcomes," *IEEE Access* 9 (2021): 138267–138282, https://dx.doi.org/10.1109/ACCESS.2021.3118010.

54. A. M. Farid and L. Ribeiro, "An Axiomatic Design of a Multi-Agent Reconfigurable Mechatronic System Architecture," *IEEE Transactions on Industrial Informatics* 11, no. 5 (2015): 1142–1155, https://doi.org/10.1109/TII.2015.2470528.

55. A. M. Farid, "Measures of Reconfigurability and Its Key Characteristics in Intelligent Manufacturing Systems," *Journal of Intelligent Manufacturing* 28, no. 2 (2017): 353–369, https://doi.org/10.1007/s10845-014-0983-7.

56. T. J. van der Wardt and A. M. Farid, "A Hybrid Dynamic System Assessment Methodology for Multi-Modal Transportation-Electrification," *Energies* 10, no. 5 (2017): 653, https://doi.org/10.3390/en10050653.

57. A. M. Farid, "Electrified Transportation System Performance: Conventional vs. Online Electric Vehicles," in *The On-Line Electric Vehicle: Wireless Electric Ground Transportation Systems*, ed., N. P. Suh and D. H. Cho (Springer, 2017), ch. 20, 279–313, https://doi.org/10.1007/978-3-319-51183-2_20.

58. I. S. Khayal and A. M. Farid, "Axiomatic Design Based Volatility Assessment of the Abu Dhabi Healthcare Labor Market," *Journal of Enterprise Transformation* 5, no. 3 (2015): 162–191, https://doi.org/10.1080/19488289.2015.1056449.

59. A. Hosseini and A. M. Farid, "A Hetero-Functional Graph Theory Perspective of Engineering Management of Mega-Projects," *IEEE Access* 13 (2025): 174559–174571, https://doi.org/10.1109/ACCESS.2025.3618284.

60. A. Hosseini and A. M. Farid, "Extending Resource Constrained Project Scheduling to Mega-Projects With Model-Based Systems Engineering & Hetero-Functional Graph Theory," preprint, arXiv, October 21, 2025, https://arxiv.org/abs/2510.19035.